

Flusso scrittura programma C++:



using namespace std: per evitare di scrivere sempre std::cout e std::endl

Tipi di errori:

- sintassi→errori nell'uso delle regole grammaticali
- logici
- errori di linker (funzione dichiarata ma non implementata)
- errori di compilazione

Operatore sizeof→lo uso in particolare per determinare quanti elementi ci sono in un array (sizeof(arr)/sizeof(arr[0]))

La programmazione struttura segue il teorema di Bohem Jacopini: ogni algoritmo può essere scritto con tre strutture di controllo (la sequenza, l'iterazione e la selezione).

Perchè il cpp:

Uno dei linguaggi più completi. Supporta costrutti quali: • Ereditarietà multipla • Puntatori • Template • Overloading degli operatori • Binding statico e dinamico • Namespace

Casting

Run-time:

- `dynamic_cast`→utilizzato per gestire il polimorfismo (il downcasting, da una classe base *Veicolo* ad una classe derivata *Auto*), è super sicuro

Compile-time:

- `static_cast`→da `double` a `int`, è eseguito a livello di compilazione tra tipi correlati
- `const_cast`→può aggiungere o rimuovere il modificatore `const` (viene usato per esempio se vogliamo passare un oggetto costante ad una funzione che un parametro non costante e sappiamo che non lo modificherà; ex: da puntatore costante a non costante, il contrario è implicito)
- `reinterpret_cast`→molto potente, effettua conversioni di basso livello direttamente sui bit

Es: `(double) somma/n` è pericoloso, si usa invece `static_cast<double> (somma)/n`

Differenza tra struct e class

Hanno entrambe il supporto per l'OOP

- i membri nella struct sono sempre public, mentre nella class sono di default private (a meno che non inserisco le keyword `public`, `protected`...) e l'ereditarietà di default è public per le struct, private per le classi
- le struct le uso come strutture dati semplici, le classi come oggetti complessi (ho dei metodi e degli stati interni)

Strutture Dati

Stack e Queues possono essere implementate sia con array che con Liste

Concatenate, quest'ultime sono preferibili perchè in questo modo la dimensione non è limitata

Le pile (o Stack) sono strutture di tipo LiFo con operazioni come `push` (inserisci l'elemento nella testa della lista) e `pop` (rimuovi e restituisci la testa della lista).

Le code sono strutture di tipo Fifo con operazioni come `enqueue` (inserisci alla fine), `dequeue` (rimuovi e restituisci il primo elemento) e `front` (restituisci il primo elemento). Notiamo che `pop` e `dequeue` hanno lo stesso comportamento.

Liste concatenate

- sono insiemi dinamici (non conosco a priori la quantità di elementi)
- rispetto all'array l'inserimento è $O(1)$ (nell'array potrei aver la necessità di ridimensionare quindi in media $O(n)$)
- la lettura sequenziale è sempre $O(1)$
- l'accesso in base all'indice è più lento dell'array ($O(n)$ vs $O(1)$)
- l'array è una sequenza omogenea e contigua di dati in memoria
- le liste concatenate sono una sequenza omogenea di dati in memoria (ma siccome alloco i nodi della lista dinamicamente, i nodi si trovano sparsi nella memoria)

Ci sono varie liste:

- liste semplici (ogni nodo è collegato al successivo)
- liste doppiamente concatenate (ogni nodo è collegato al precedente e al successivo)
- liste circolari semplici (l'ultimo nodo è anche collegato al primo nodo)
- liste circolari doppiamente concatenate (l'ultimo nodo è anche collegato al primo nodo e viceversa)

LinkedList: lo implemento in genere come una classe, contiene il valore effettivo e un puntatore al nextNode

A volte per un'implementazione più semplice delle liste si può inizializzare la head/startNode con un dummy node.

Per le code, per aggiungere un elemento alla fine della lista si può memorizzare l'ultimo nodo man mano e così facendo si mantiene la complessità $O(1)$ (sennò dovremmo ogni volta cercare l'ultimo nodo e avremmo $O(n)$), bisogna fare attenzione a mantenere l'integrità della struttura.

Programmazione OOP

- classi, insieme di oggetti che condividono struttura e comportamento (quindi attributi e metodi, cioè funzioni dentro le classi)
per definire una classe, dichiariamo gli attributi (in genere come private) e le funzioni (i **metodi**) che poi possiamo implementare dentro la classe (sono trattati come funzioni inline) oppure fuori.
- specificatori di accesso:
 - private→solo all'interno della classe
 - protected→anche da classi derivate da questa
 - public→da tutti
- gli oggetti sono istanze di classi
- **Specificatore di ereditarietà nel costruttore della sottoclasse (private, protected, public) e nella definizione delle sottoclassi**

- astrazione (semplifichiamo la realtà concentrandoci solo sugli elementi importanti del sistema complesso che stiamo studiando)
- ereditarietà, magari ci sono più classi che ereditano caratteristiche (attributi, metodi) da una superclasse; in questo caso io posso ridefinire alcuni metodi(override) o estendere la superclasse. (introduco i concetti di superclasse e sottoclasse)

C++ supporta l'ereditarietà multipla, rispetto a Java che supporta l'ereditarietà singola e implementa una sorta di ereditarietà multipla con le interfacce; l'ereditarietà multipla può infatti portare a situazioni inconsistenti, esempio: ho due o più metodi nella superclasse con lo stesso nome e il compilatore non riesce a capire quale chiamare nonostante l'overloading delle funzioni vedi esempio FiguraMagica, dove i metodi getX e getY risultavano duplicati e quindi devo per forza usare la notazione figuraMagica.Cerchio::getX()

- incapsulamento (information hiding) e occultamento→le classi “incapsulano” metodi e attributi; io nascondo gli attributi agli sviluppatori esterni e, per quanto possibile, espongo metodi che mi consentono di modificare gli attributi della classe (=lo stato interno della classe) senza perdere l'integrità della classe
- polimorfismo, più forme, un metodo definito in una superclasse che viene poi ridefinito nelle sottoclassi

parlando di polimorfismo si introducono i concetti di binding statico e dinamico:

- nel linguaggio C il binding è praticamente solo statico (ad una funzione corrisponde uno ed un solo blocco di codice), risolto a compile-time
- nel linguaggio C++ il binding può essere dinamico, l'associazione oggetto-metodo corretto viene eseguita a runtime (il compilatore infatti crea una virtual table così a run time sa quale funzione chiamare in base alla classe dell'oggetto, questo genera un leggero overhead, tollerabile) per indicare al compilatore di usare il binding dinamico inserisco la keyword virtual in ogni funzione su cui voglio fare l'override (conviene anche nelle derivate, perchè virtual funziona solo sul primo livello di sottoclassi)

Rispetto alla sola programmazione procedurale (dove contano le procedure, gli algoritmi), nella programmazione orientata agli oggetti conta soprattutto il sistema, abbiamo un programma che è sicuramente modulare e che rispecchia meglio la realtà.

Per dichiarare una classe astratta (cioè non istanziabile) in c++ basta che vi sia almeno una funzione virtuale pura (definita con = 0), che le classi figlie devono obbligatoriamente implementare (provando ad istanziare una classe astratta otteniamo l'errore *unimplemented pure virtual method in ...*)

Programmazione generica

Posso usare i template per definire classi funzioni che lavorano con variabili/metodi senza specificare il loro tipo (che diventa un generico **T** o **V**).

I vectors di Cpp per esempio funzionano proprio con le classi generiche.

Un template è un modello per istanziare classi o funzioni del template a tempo di compilazione.

Per usare i tipi generici **T** e **V** devo specificare di star usando un template, per ogni metodo o classe devo scrivere prima dell'utilizzo del tipo generico

template <typename T>

Così il compilatore è come se per ogni tipo che io scelgo, crea più copie di funzione basandosi sul modello template.

I template (o generics) consentono così di astrarre il tipo.

Ricorsione

Ricorsione e induzione sono praticamente la stessa cosa.

Una funzione che richiama se stessa è definita ricorsiva (come esempi vedi il fattoriale, la funzione di fibonacci che sono funzioni ricorsive matematiche, ma ci sono tanti algoritmi ricorsivi come la ricerca binaria, gli algoritmi divide et impera, tanti algoritmi sui grafi).

Un'osservazione molto importante: ogni algoritmo scritto in modo ricorsivo può essere scritto in modo iterativo e viceversa.

Ciò è valido anche nei casi apparentemente più complessi (algoritmi sui grafi o algoritmi di sorting) perchè la ricorsione utilizza una struttura dati ben specifica (lo stack) che possiamo implementare con liste concatenate.

Quando definiamo un algoritmo ricorsivo dobbiamo sempre definire un caso base e la chiamata ricorsiva (senza il caso base abbiamo uno stackoverflow, una ricorsione infinita).

La ricorsione consente quindi di scrivere rapidamente algoritmi più semplici da leggere, ma che rispetto alla controparte iterativa sono più lenti (c'è infatti l'overhead della chiamata ricorsiva, che costruisce un nuovo frame/record di attivazione), c'è un maggior spreco di risorse (sia memoria che tempo).

Siccome però ogni programma ricorsivo può poi essere scritto con un'apposita struttura dati (lo stack), risulta più semplice poi scrivere la soluzione iterativa.

ricorsione indiretta: una funzione richiama se stessa non direttamente ma tramite chiamate ad altre funzioni.

Overheading/Overloading delle funzioni

Qualcuno lo chiama polimorfismo, ma non è tecnicamente corretto. In Cpp posso avere più funzioni con lo stesso nome, e il compilatore le può differenziare in base al numero e al tipo degli argomenti (non in base al tipo di ritorno).

Overloading degli operatori

Posso sovraccaricare i vari operatori unari e binari, esempio classico per gli stream:

```
template <typename T>
ostream &operator<<(ostream &s, List<T> &list)
{
    ListNode<T> *iterator = list.getStartNode();
    while (iterator != nullptr)
    {
        s << iterator->getValue() << " ---> ";
        iterator = iterator->getNextNode();
    }
    s << "NULLPTR";
    return s;
}
```

Posso fare l'overloading degli operatori tramite metodo interno alla classe o tramite una funzione (esterna alla classe):

- in certi casi (vedi streams) non posso implementarlo come metodo in classe perchè dovrei modificare le standard library; per accedere agli attributi dell'oggetto ci dovrebbero essere disponibili i metodi getters, sennò posso usare la keyword friend

Metodo interno alla classe:

```
bool operator==(OraDelGiorno& other) {
return h_==other.h_ && m_==other.m_ && s_==other.s_;
}
```

Funzione esterna alla classe

```
bool operator==(OraDelGiorno& first, OraDelGiorno& other) {
return first.getHour()==other.getHour() && first.getMinutes()==other.getMinutes() &&
first.getSeconds()==other.getSeconds();
}
```

Reference vs Puntatori

Di default in Cpp il passaggio di variabili nelle funzioni è per valore (le variabili vengono copiate), spesso è necessario fare il passaggio per riferimento usando i puntatori, o le reference (sono intercambiabili, le reference sono puntatori costanti, rispetto ai puntatori migliorano la leggibilità):

- variabile molto grande da copiare
- devo necessariamente modificare la variabile passata (specialmente per le liste concatenate e quando eseguo l'overloading degli operatori stream)

Nelle funzioni posso anche restituire references.

Puntatori a costante e puntatori costanti

Puntatori a costante **const int* ptr**: non posso modificare il valore della variabile puntata, ma posso modificare l'indirizzo

Puntatori costanti **int* const ptr**: non posso modificare l'indirizzo, ma posso modificare il valore della variabile puntata

Utilizzo di gdb

Il Gnu Developer Debugger consente di eseguire il codice passo passo, di fermarlo in punti scelti (breakpoint), mostrare come cambiano le variabili a runtime. Per utilizzarlo bisogna aggiungere **-g** prima del nome del sorgente del programma.

Comandi del gdb (si possono vedere scrivendo help):

- run→esegue il programma
- c→continua al prossimo break point
- s→step, esegui entrando nelle funzioni
- n→next, esegui senza entrare nelle funzioni

Per controllare i memory leaks si possono usare su Linux valgrind e address sanitizer.

Funzioni inline e funzioni template

Le funzioni template sono sempre funzioni inline. Le funzioni inline vengono ottimizzate dal compilatore nel senso che il loro codice viene realmente copiato in ogni punto in cui la funzione è invocata (è in realtà il compilatore che decide se conviene o non conviene farlo). Le funzioni ricorsive ovviamente non possono essere inline. Dichiarare e implementare i metodi dentro le classi produce metodi inline.

Esempio di funzione template

```
template <typename T>
```

```
T max (T a, T b) {  
return a > b ? a : b;  
}
```

Stringhe

Le stringhe in C++ sono realizzate in una classe apposita (#include <string>):

- gestione dinamica della memoria
- overloading degli operatori = (assegnamento), e del confronto ==, <=, >=, <, >
- accesso ai singoli caratteri con [i]
- metodi interessanti come length, c_string, find(str)

Allocazione dinamica memoria

Se in C l'allocazione dinamica era con malloc, calloc, realloc e free.

In Cpp utilizziamo le keyword new e delete.

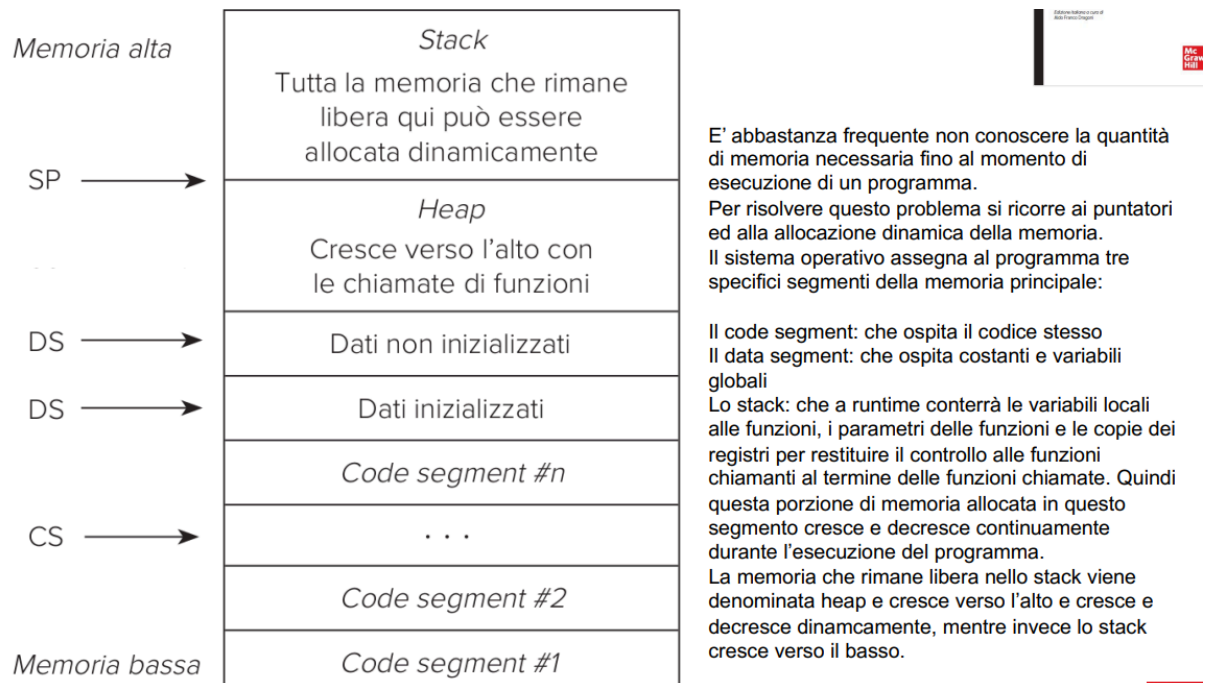
New ci consente di istanziare oggetti nell'heap fornendo il valore delle variabili per il costruttore:

- tipo* puntatore = new tipo // non array

- `tipo* puntatore = new tipo[dimensione] // array`

Ci sono due tipi di delete, `delete obj` e `delete [] arr`.

È importante usare il delete per liberare la memoria ed evitare memory leaks (alla chiusura del programma la memoria viene liberata dal sistema operativo, ma per applicazioni che realisticamente non vengono mai chiuse ciò non accadrà mai).



Lavorare con i file

In Cpp per lavorare con l'I/O posso usare gli stream. Lo stream è un'astrazione che rappresenta un flusso di dati. Possono essere di input o di output.

Sono disponibili nella standard library questi header:

- `iostream` → classi `ostream`, `istream` e `iostream` per le operazioni di I/O basilari, e gli oggetti `cout`, `cin`, `cerr`, `clog`
- `fstream` → classi `ifstream`, `ofstream`, `fstream` per operazioni su file
- `sstream` → classi `istringstream` e `ostringstream`
- `strstream` → classi `istrstream`, `ostrstream`, `strstream` per formattare dati

Per verificare se ci sono stati problemi di lettura/scrittura in uno stream uso

`cin.good()`, `cin.fail()`, `cin.bad()`

`ifstream fin ("demo")`

- apri il file in lettura (verificare con `good` se è tutto apposto)

`ofstream fout ("demo", ios::out)`

- sovrascrivi il file (verificare con `good` se è tutto apposto)

Vita di un file → Il file viene aperto in lettura/scrittura o tutti e due, vengono effettuate le operazioni e poi è fondamentale chiudere lo stream.

Inizializzare gli attributi in una classe

- con il vettore di inizializzazione

- dentro il costruttore con this->
- dal C++11 posso direttamente dichiarare la variabile e inizializzarla

Istanziamento di un oggetto

- nello stack (viene poi deallocato automaticamente appena va out of context):
Cerchio cerchio(1,1,4);
- nell'heap (viene mantenuto anche quando va out of context, deve essere liberato dallo sviluppatore, altrimenti memory leaks): new Cerchio(1,1,4)

Distruttore ~

- consente di verificare se per sbaglio abbiamo creato shallow copies (se abbiamo fatto shallow copies per sbaglio, il distruttore verrà chiamato più volte)
- non ha parametri e nemmeno un tipo di ritorno
- viene chiamato solo indirettamente quando usiamo delete, oppure quando la variabile va out of scope
- serve a deallocare correttamente la memoria (vedi come esempio la lista concatenata)
- è complementare del costruttore
- se ho ereditarietà, per evitare memory leaks metto virtual nella classe base
- ordine speculare, ex: ho la gerarchia Veicolo→Motore→Auto
e voglio un'istanza di Auto, prima viene chiamato il costruttore di Veicolo, poi di Motore, poi di Auto. voglio fare il delete, prima viene chiamato il distruttore di Auto, poi di Motore e poi di Veicolo.

Vectors

I std::vector in cpp sono array dinamici:

- memoria contigua (quindi supportano l'accesso casuale)
- dinamici (gestiscono autonomamente la riallocazione della memoria)

Utilizzo:

```
#include <vector>
```

```
std::vector<int> v = {1, 2, 3}; // Inizializzazione
```

```
v.push_back(4); // Aggiunge 4 alla fine
```

```
v.pop_back(); // Rimuove l'ultimo elemento
```

```
v.size(); // Quanti elementi ci sono? (3)
```

```
v.clear(); // Svuota il vector
```

```
v.empty(); // Restituisce true se è vuoto
```

```
// Iterazione moderna (C++11+)
```

```
for (int x : v) {
```

```
// Fai qualcosa con x
}
```

Complessità

Il costo di un algoritmo dipende da tempo (quante operazioni devo eseguire) e spazio (in memoria RAM).

Esempi:

- costo di un if (condizione) then yes() else no()
 $costo(if) = costo(condizione) + \max(costo(yes), costo(no))$
- for(i = 0; i < n; i++) {cose_belle()}

$$costo(for) = \sum_{i=0}^n costo(cose_belle())$$

- while (condizione) {cose_belle()}

$$costo(while) = \sum_{i=0}^n (costo(cose_belle()) + costo(condizione))$$

- costo di una funzione non ricorsiva: istanziare il frame, spostare i valori e poi l'esecuzione effettiva

Non ci interessa una specifica macchina, ma il caso generale quindi il Tempo impiegato proporzionale al numero di operazioni eseguite.

Inoltre, individuare una funzione che consente di calcolare precisamente quante operazioni vengono effettuate per un determinato algoritmo è molto complesso, ci interessiamo quindi solo alla notazione asintotica, calcoliamo come spazio e tempo (inteso come numero di operazioni) crescono al variare della dimensione dell'input.

Big O → descrive un upper bound (il worst case), il tempo massimo di esecuzione

Big Ω → descrive un lower bound (il best case), l'esecuzione più veloce possibile

Big Θ → descrive un tight bound (**quando Big O e Big Ω coincidono**), il numero di operazioni varia allo stesso modo sia per il worst case che per il best case

Esempi:

- Bubble sort, $O(n^2)$ ma $\Omega(n)$ quindi la Big Θ non è definita
- Merge sort, $O(n \log n)$ e $\Omega(n \log n)$ e $\Theta(n \log n)$, quindi esegue la stessa quantità di operazioni sia per il worst case che per il best case

Definiamo la **Big O** a meno di una costante:

$$O(g(n)) = \{f(n) \text{ se } \exists c, v > 0 \text{ tali che } \forall n > v \text{ si ha che } 0 \leq f(n) \leq cg(n)\}$$

praticamente ci concentriamo solo su come l'algoritmo scala, ignorando dettagli irrilevanti (come costanti o termini più piccoli).

Con la Big O gli algoritmi vengono divisi in classi di equivalenza, nella stessa classe troviamo tutti gli algoritmi la cui complessità asintotica è dello stesso ordine di grandezza:

- $O(1)$ → inserimento in una lista concatenata

- $O(\log n)$ → ricerca binaria
- $O(n)$ → ricerca lineare
- $O(n \log n)$ → merge sort e quick sort
- $O(n^2)$ → insertion sort, selection sort, bubble sort (due cicli annidati)

La **Big Ω** fornisce un lower bound:

$f(n) = \Omega(g(n))$ se esistono $c, v > 0$ tali che $f(n) \geq cg(n)$ per $\forall n > v$

Osservazioni:

- **Big O** Se un algoritmo è $O(n^3)$, allora è "per forza" anche $O(n^4)$ o $O(n^5)$
- **Big Ω** Se un algoritmo è $\Omega(n)$, significa che è anche $\Omega(\log n)$ o $\Omega(1)$
- **Big Θ** Se le due O coincidono, abbiamo una condizione forte

Analogia "Il prezzo di un computer":

- O-grande: "Costa al massimo 1000€". (Non spenderai di più, ma potresti spenderne 500€).
- **Omega (Θ)**: "Costa **almeno** 800€". (Non spenderai di meno, ma potresti spenderne 2000€).
- **Theta (Ω)**: "Il prezzo è **compreso tra** 800€ e 1000€". Hai delimitato con precisione la fascia di prezzo.

Un algoritmo A che risolve un problema P si dice ottimale se P ha complessità $\Omega(f(n))$ ed A ha complessità $O(f(n))$. Vedi l'ordinamento per confronto $O(n \log n)$

Analisi di algoritmi con cicli for

- le varie Big O si moltiplicano tra di loro
- se l'incremento i è lineare, ho $O(n)$
- se l'incremento i è moltiplicato per una costante ho $O(\log n)$
- se l'incremento i è modificato con una potenza (pow) ho $O(\log \log n)$

Complessità per funzioni ricorsive

Paradigma divide et impera (divide and conquer): risolviamo un problema con la ricorsione applicando tre passaggi (dividi il problema in sottoproblemi, conquista → risolvi i sottoproblemi ricorsivamente, combina le soluzioni dei sottoproblemi ottenendo la soluzione del problema originale)

Per studiare la complessità delle funzioni ricorsive utilizzo strumenti matematici come le sommatorie, e l'induzione ($P(x)$ vale per il caso base, allora supposto vera l'ipotesi induttiva per n , verifico che vale anche per $n+1$).

Per studiare gli algoritmi ricorsivi cerchiamo dei pattern nello schema di ricorsione:

- $T(n) = T(n - 1) + n \Rightarrow T(n)$ è circa $\frac{n^2}{2}$ (l'algoritmo procede "eliminando" un elemento alla volta). Es: selection sort
- $T(n) = T(\frac{n}{2}) + 1 \Rightarrow T(n)$ è circa $\ln(n)$ (l'algoritmo procede dimezzando l'input ad ogni passo). Es: Binary search

- $T(n) = T(\frac{n}{2}) + n \Rightarrow T(n)$ è circa $2n$ (l'algoritmo procede dimezzando l'input e esaminando ogni elemento di esso, ad ogni passo). Es: quickselect
- $T(n) = 2T(\frac{n}{2}) + n \Rightarrow T(n)$ è circa $n \log(n)$ (l'algoritmo esegue una scansione lineare dell'input prima, durante, oppure dopo aver suddiviso l'input in due parti)
- $T(n) = 2T(\frac{n}{2}) + 1 \Rightarrow T(n)$ è circa $2n$ (l'algoritmo dimezza l'input ed esegue una quantità di lavoro addizionale costante)
- (...ci sono altre serie che devo ricordare?)

Sorting algorithms

Algoritmi $O(n^2)$ in media:

- insertion sort (sposta gli elementi indietro, best case $O(n)$ ma medium-case e worst case $O(n^2)$)
- selection sort, una ricerca lineare (per trovare il minimo) eseguita n volte: $\Theta(n^2)$
- bubble sort (confronto elementi adiacenti e faccio risalire il maggiore alla fine di ogni iterazione, dopo $n-1$ iterazioni l'array è ordinato, caso migliore $O(n)$, caso peggiore e caso medio sono $O(n^2)$)

Tecnica divide and conquer:

- è una tecnica prettamente ricorsiva
- si basa sul dividere il problema in sottoproblemi, risolvere i sottoproblemi, combinare le soluzioni ottenendo la soluzione al problema originale

Merge sort: dividiamo ricorsivamente l'array in sottoarray e arriviamo al caso base già risolto (l'array che contiene un singolo elemento è già ordinato), ricorsivamente quindi utilizziamo la procedura merge che unisce due array ordinati in un unico array ordinato

- analisi complessità: la procedura merge ha complessità $O(n)$, l'equazione di ricorrenza in questo caso è $T(n) = 2T(n/2) + n = O(n \log n)$
- ma possiamo dire di più, il mergesort ha complessità proprio $\Theta(n \log n)$

Funzionamento metodo merge:

- siccome i due sottoarray sono ordinati, posso costruire un array ordinato: tengo un ciclo e due puntatori ai primi elementi dei due array, man mano scelgo l'elemento più piccolo da i due array e avanzo il puntatore corrispondente
- alla fine uno dei due array si sarà svuotato, inserisco gli elementi dell'altro array e faccio il mem copy per copiare gli elementi nell'array principale

Quicksort: partizioniamo l'array in base ad un pivot, che può essere scelto a caso, nella parte sinistra ci sono gli elementi minori del pivot, poi il pivot, poi gli elementi maggiori del pivot, applichiamo ricorsivamente l'algoritmo sulle due partizioni di destra e sinistra

- analisi complessità, la funzione partition richiede $O(n)$, l'equazione di ricorrenza nel caso medio è la stessa del merge sort $T(n) = 2T(n/2) + n = O(n \log n)$ il caso peggiore però è $O(n^2)$, quando le due partizioni create dal pivot sono completamente sbilanciate (0 e $n - 1$), ho scelto o il primo o l'ultimo elemento come pivot. è molto raro come caso e per evitarlo basta prendere il pivot randomicamente. inoltre, anche se il partizionamento avviene 20%-80% oppure 10%-90% si può comunque dimostrare che la complessità media rimane $O(n \log n)$.

notiamo che, pur avendo in media la stessa complessità del mergesort, il quicksort è più veloce perchè, rispetto al mergesort, non richiede la creazione di un nuovo array e la copiatura dal nuovo array all'array principale, ma fa tutto in place.

mergesort: prima passo ricorsivo, poi merge

quicksort: prima partizione, poi passo ricorsivo

Ricerca

	Best case	Medium case	Worst Case
Ricerca binaria (solo se l'array è ordinato)	$O(1)$	$O(\log n)$	$O(\log n)$
Ricerca lineare	$O(1)$	$O(n)$	$O(n)$

Utilizzo del const